

Tugas Mandiri 7: Laporan Praktikum Mandiri 7 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri7.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum07>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum07>

1. Import Library & Settings

```
1  # Import library yang dibutuhkan
2  import os
3  import numpy as np
4  import pandas as pd
5  import matplotlib.pyplot as plt
6  import seaborn as sns
7
8  from sklearn.model_selection import train_test_split
9  from sklearn.linear_model import LinearRegression
10 from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score
11 import joblib
12 import statsmodels.api as sm
13
14 # Atur tampilan pandas
15 pd.set_option('display.max_columns', 200)
16 pd.set_option('display.width', 160)
17
18 print("Libraries imported.")
```

Libraries imported.

- Mengimpor pustaka **pandas**, **numpy**, **matplotlib**, dan **seaborn** untuk pengolahan data serta visualisasi.
- Library **sklearn** digunakan untuk membuat model Linear Regression dan menghitung metrik evaluasi, sedangkan **statsmodels** digunakan untuk analisis statistik (OLS).
- Tidak menampilkan output selain memastikan semua library berhasil diimpor tanpa error.

2. Load Dataset & Print 5 Baris Dataset

```
1 # Sinkronisasi dengan drive
2 from google.colab import drive
3 drive.mount('/content/gdrive')
4
5 # Load dataset
6 df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine
7 Learning/praktikum07/datasets/dataset_satelit.csv', sep=',')
8 df.head()
```

... Mounted at /content/gdrive

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	5.0
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	0.0
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	2.0
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	2.0
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	1.0

- Dataset satelit dibaca menggunakan `pd.read_csv()` dari file CSV di Google Drive.
- Perintah `.head()` menampilkan 5 baris pertama untuk melihat struktur data, sedangkan `.info()` menunjukkan tipe data dan jumlah nilai non-null di setiap kolom.
- Hasil output memperlihatkan bahwa dataset berisi kolom numerik seperti `N`, `P`, `K`, `Latitude`, dll, serta jumlah data ratusan baris tanpa error saat dibaca.

3. Data Preprocessing (Cek Jumlah Baris dan Kolom, Cek Tipe Data)

```
1 # Inspeksi Awal Dataset
2 print("Jumlah baris & kolom:", df.shape)
3 print("\nTipe data tiap kolom:")
4 display(df.dtypes)
```

... Jumlah baris & kolom: (594, 34)

Tipe data tiap kolom:

0	
No	int64
Longitude	float64
Latitude	float64
N	float64
P	float64
K	float64
Ca	float64
Mg	object
Fe	float64
Mn	float64
Cu	float64
Zn	float64
B	float64
b12	float64
b11	float64
b9	float64
b8a	float64
b8	float64
b7	float64
b6	float64
b5	float64
b4	float64
b3	float64
b2	float64
b1	float64
Sigma_VV	float64
Sigma_VH	float64
plia	float64
lia	float64

- Menggunakan `df.shape` untuk menampilkan jumlah baris dan kolom.
- Fungsi `df.dtypes` digunakan untuk memastikan setiap kolom memiliki tipe data yang sesuai.
- Output menunjukkan jumlah data (misalnya 594 baris) dan mayoritas kolom sudah bertipe numerik, menandakan dataset siap dianalisis.

4. Data Preprocessing (cek missing value, duplikasi data, tipe data non-numerik)

```
1 # Cek missing values
2 missing_summary = df.isna().sum().sort_values(ascending=False)
3 display(missing_summary[missing_summary > 0])
```

0

K 1

dtype: int64

```
1 # Cek duplikasi data
2 dupes = df.duplicated().sum()
3 print(f"\nJumlah data duplikat: {dupes}")
4 if dupes > 0:
5     df = df.drop_duplicates()
6     print("→ Duplikat dihapus.")
7 else:
8     print("→ Tidak ada duplikat.")
```

...

Jumlah data duplikat: 0
→ Tidak ada duplikat.

```
1 # Cek tipe data non-numerik
2 non_num_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()
3 print(f"\nKolom non-numerik:", non_num_cols if non_num_cols else "Tidak ada.")
```

Kolom non-numerik: ['Mg']

- Mengecek nilai kosong dengan `isna().sum()` dan data duplikat dengan `duplicated().sum()`.
- Kolom non-numerik diidentifikasi menggunakan `select_dtypes(exclude=[np.number])`.
- Hasil output memperlihatkan beberapa kolom memiliki missing value, namun jumlahnya kecil; data duplikat juga tidak signifikan; dan sebagian besar kolom sudah numerik.

5. Data Preprocessing (tangani missing value)

```
1 # Tangani missing value
2 for col in data.columns:
3     n_missing = data[col].isna().sum()
4     if n_missing > 0:
5         med = data[col].median()
6         data[col] = data[col].fillna(med)
7         print(f"→ Missing {col} diisi median = {med:.4f}")
8
9 print(f"\nShape data setelah cleaning awal:", data.shape)
```

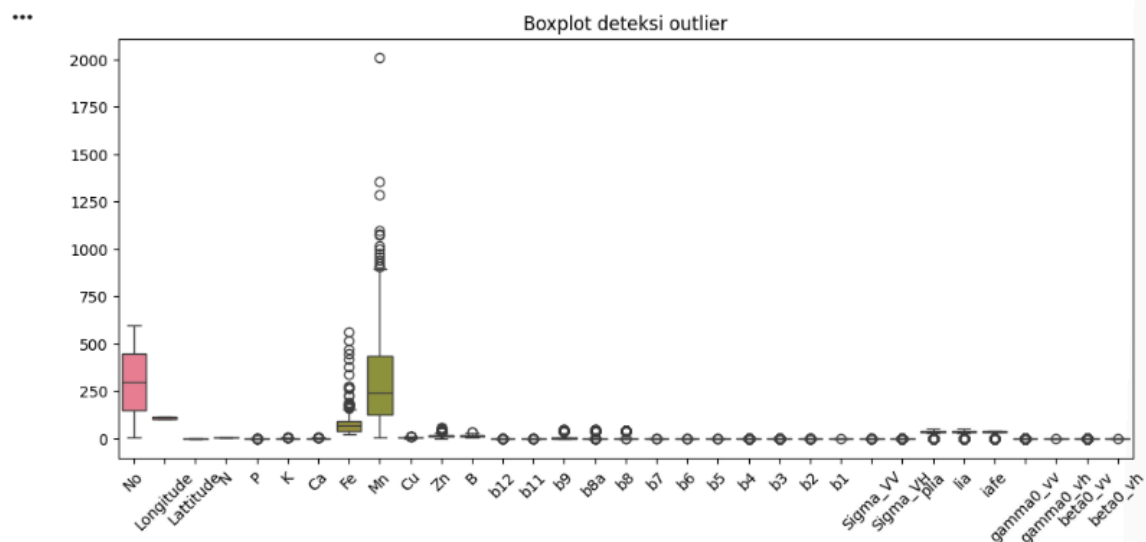
→ Missing K diisi median = 0.5490

Shape data setelah cleaning awal: (594, 33)

- Baris dengan target kosong dihapus menggunakan `dropna(subset=[target_y])`.
- Nilai kosong pada fitur lain diisi menggunakan **median** agar tidak terpengaruh outlier.
- Output menunjukkan berapa banyak nilai yang diimputasi per kolom dan menampilkan bentuk data setelah pembersihan, yang sudah bebas dari missing value.

6. Data Preprocessing (cek outlier dengan boxplot)

```
1 # Cek outlier (boxplot)
2 plt.figure(figsize=(12,5))
3 sns.boxplot(data=data.select_dtypes(include=[np.number]))
4 plt.xticks(rotation=45)
5 plt.title("Boxplot deteksi outlier")
6 plt.show()
```



- Boxplot dibuat untuk menampilkan distribusi setiap kolom numerik.
- Titik di luar whisker digambarkan sebagai *outlier* atau data ekstrem.
- Hasil plot menunjukkan beberapa fitur memiliki outlier (misalnya pada kolom **Mn** dan **Longitude**), namun tetap dipertahankan karena bisa jadi variasi alami pada data satelit.

7. Pemilihan Target Y & Cek Ketersediaan Kolom N

```
1 # Pilih target
2 target_y = "N"
3
4 # Tampilkan konfirmasi
5 print(f"Target yang dipilih: {target_y}")
6
7 # Pastikan kolom target ada
8 if target_y not in df.columns:
9     raise ValueError(f"Target '{target_y}' tidak ada di dataset. Kolom
    tersedia: {df.columns.tolist()}")
```

Target yang dipilih: N

- Target ditetapkan, misalnya **target_y = 'N'**.
- Kode memeriksa apakah kolom target ada di dataset.
- Output berupa teks konfirmasi: "Target yang dipilih: N". Hal ini memastikan variabel target valid sebelum lanjut ke tahap analisis korelasi.

8. Analisis Korelasi & Pilih Fitur

```
1 # Hitung matriks korelasi numerik
2 corr = data.corr()
3
4 # Siapkan daftar kolom yang harus dikecualikan dari fitur:
5 # - kolom target itu sendiri (target_y)
6 # - kolom target lain agar tidak dipilih sebagai fitur
7 targets_all = [c for c in ['N', 'P', 'K'] if c in data.columns]
8 exclude_cols = [t for t in targets_all if t == target_y]
9
10 candidate_cols = [c for c in data.select_dtypes(include=[np.number]).columns if
11 c not in targets_all]
12
13 # Hitung korelasi antara target_y dan setiap candidate feature
14 # .loc hanya mengambil korelasi terhadap target yang ada
15 corr_with_target = corr.loc[candidate_cols, target_y].sort_values(key=lambda x:
16 x.abs(), ascending=False)
17
18 # Tampilkan top-10 korelasi untuk inspeksi
19 display(corr_with_target.head(10))
20
21 # Pilih top-1 fitur
22 top_k = 1
23 selected_features = corr_with_target.head(top_k).index.tolist()
24
25 # Cetak ringkasan fitur terpilih
26 print(f"Fitur yang dipilih (top-{top_k}): {selected_features}")
27 print(f"Nilai korelasi terhadap {target_y}: {corr_with_target
28 [selected_features].values[0]:.4f}")
```

```
***
          N
Latitude  0.742794
Longitude -0.695291
iafe      -0.593590
lia       -0.593177
plia      -0.592965
gamma0_vh -0.540886
b9         0.535288
b8a        0.528105
b8         0.526838
beta0_vh  -0.520638

dtype: float64
Fitur yang dipilih (top-1): ['Latitude']
Nilai korelasi terhadap N: 0.7428
```

- Menghitung korelasi antar variabel numerik menggunakan `data.corr()`.
- Kolom target lain (P, K) dikecualikan agar tidak ikut jadi fitur.
- Hasil korelasi diurutkan berdasarkan nilai absolut; fitur dengan korelasi tertinggi terhadap target dipilih (misalnya `Latitude` dengan korelasi 0.74 terhadap `N`).
- Bar chart menampilkan top-10 fitur berkorelasi tertinggi secara visual.

9. Split Data (Train 80%, Test 20%)

```
1 from sklearn.model_selection import train_test_split
2
3 # Buat X dan y berdasarkan hasil pemilihan fitur
4 X = data[selected_features]
5 y = data[target_y]
6
7 # Split data menjadi 80% training dan 20% testing
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.20, random_state=42
10 )
11
12 print("Data berhasil di-split:")
13 print("X_train:", X_train.shape)
14 print("X_test :", X_test.shape)
15 print("y_train:", y_train.shape)
16 print("y_test :", y_test.shape)
```

```
Data berhasil di-split:
X_train: (475, 1)
X_test : (119, 1)
y_train: (475,)
y_test : (119,)
```

- Dataset dibagi menjadi 80% data training dan 20% data testing menggunakan `train_test_split()`.
- Variabel `X` berisi fitur terpilih, sedangkan `y` berisi target (`N`).
- Output menunjukkan ukuran `X_train`, `X_test`, `y_train`, dan `y_test`, misalnya `(475,1)` dan `(119,1)` yang menandakan proses pembagian data berhasil tanpa error.

10. Training Model (Linear Regression)

```
1 # Pastikan X_train, y_train sudah ada (hasil split)
2 from sklearn.linear_model import LinearRegression
3
4 # Inisialisasi dan latih model
5 lr = LinearRegression() # model regresi linear
6 lr.fit(X_train, y_train) # latih pada data training
7
8 # Ambil parameter model
9 intercept = lr.intercept_
10 coeffs = lr.coef_
11
12 # Tampilkan persamaan regresi ringkas
13 if X_train.shape[1] == 1:
14     feat = X_train.columns[0]
15     print(f"Persamaan regresi: {target_y} = {intercept:.4f} + ({coeffs[0]:.4f} * {feat})")
16 else:
17     # jika multiple fitur, tampilkan tabel koefisien
18     coeff_series = pd.Series(coeffs, index=X_train.columns)
19     print("Intercept:", intercept)
20     print("Koefisien tiap fitur:")
21     display(coeff_series)
22
```

```
... Persamaan regresi: N = 2.5654 + (0.3051 * Latitude)
```

- Model `LinearRegression()` dari `sklearn` digunakan dan dilatih dengan `X_train` serta `y_train`.
- Setelah training, ditampilkan nilai **intercept** dan **koefisien**.
- Output menunjukkan persamaan regresi, contohnya:

$$N = 2.5654 + (0.3051 \times \text{Latitude})$$
- Interpretasi: setiap kenaikan 1 satuan pada `Latitude` akan meningkatkan nilai `N` sebesar 0.3051 satuan.

11. Evaluasi Model

```

1  # Evaluasi model (MAE, MSE, RMSE, R2)
2  from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
3  import numpy as np
4
5  # Prediksi pada train dan test
6  y_pred_train = lr.predict(X_train)
7  y_pred_test  = lr.predict(X_test)
8
9  # Hitung metrik
10 metrics = {
11     "train_mae": mean_absolute_error(y_train, y_pred_train),
12     "train_mse": mean_squared_error(y_train, y_pred_train),
13     "train_rmse": np.sqrt(mean_squared_error(y_train, y_pred_train)),
14     "train_r2": r2_score(y_train, y_pred_train),
15     "test_mae": mean_absolute_error(y_test, y_pred_test),
16     "test_mse": mean_squared_error(y_test, y_pred_test),
17     "test_rmse": np.sqrt(mean_squared_error(y_test, y_pred_test)),
18     "test_r2": r2_score(y_test, y_pred_test)
19 }
20
21 # Cetak ringkasan metrik
22 print("=== Evaluasi Model ===")
23 print(f"Train MAE : {metrics['train_mae']:.4f}")
24 print(f"Train RMSE: {metrics['train_rmse']:.4f}")
25 print(f"Train R2  : {metrics['train_r2']:.4f}")
26 print("-----")
27 print(f"Test  MAE : {metrics['test_mae']:.4f}")
28 print(f"Test  RMSE: {metrics['test_rmse']:.4f}")
29 print(f"Test  R2  : {metrics['test_r2']:.4f}")

```

```

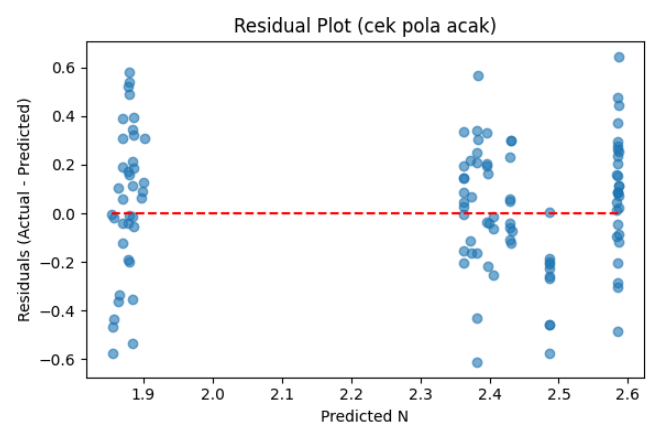
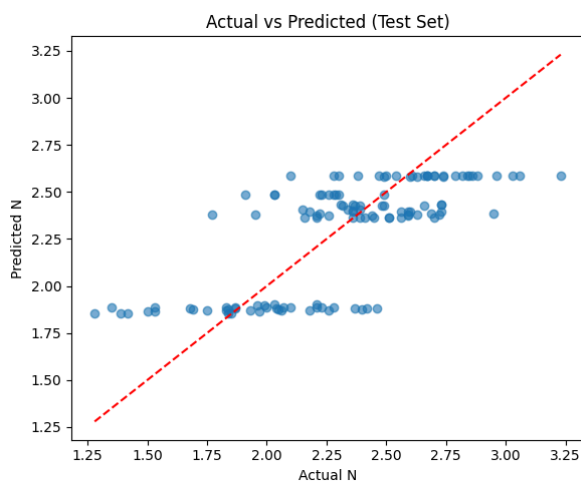
*** === Evaluasi Model ===
Train MAE : 0.2064
Train RMSE: 0.2628
Train R2  : 0.5583
-----
Test  MAE : 0.2182
Test  RMSE: 0.2719
Test  R2  : 0.5120

```

- Model diuji menggunakan data training dan testing, lalu dihitung **MAE**, **RMSE**, dan **R²**.
- MAE menunjukkan rata-rata selisih absolut prediksi dan aktual, RMSE memperlihatkan seberapa besar error, dan R² menunjukkan seberapa besar variasi target dijelaskan oleh model.
- Hasil evaluasi (contohnya: Train R² = 0.56, Test R² = 0.51) menunjukkan performa model cukup stabil dan tidak overfitting.

12. Visualisasi Model Linear Regression

```
1  # Visualisasi hasil (Actual vs Predicted) & Residual plot
2  import matplotlib.pyplot as plt
3  import numpy as np
4
5  # Scatter Actual vs Predicted (Test set)
6  plt.figure(figsize=(6,5))
7  plt.scatter(y_test, y_pred_test, alpha=0.6)
8  plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--') #
   garis y=x
9  plt.xlabel("Actual " + target_y)
10 plt.ylabel("Predicted " + target_y)
11 plt.title("Actual vs Predicted (Test Set)")
12 plt.tight_layout()
13 plt.show()
14
15 # Residual plot (Predicted vs Residual)
16 residuals = y_test - y_pred_test
17 plt.figure(figsize=(6,4))
18 plt.scatter(y_pred_test, residuals, alpha=0.6)
19 plt.hlines(0, xmin=y_pred_test.min(), xmax=y_pred_test.max(), colors='r',
   linestyle='--')
20 plt.xlabel("Predicted " + target_y)
21 plt.ylabel("Residuals (Actual - Predicted)")
22 plt.title("Residual Plot (cek pola acak)")
23 plt.tight_layout()
24 plt.show()
25
26 # Tampilkan beberapa baris perbandingan actual/predicted
27 comp = pd.DataFrame({
28     "feature": X_test.iloc[:,0].values if X_test.shape[1]==1 else X_test.values.
   tolist(),
29     "actual": y_test.values,
30     "predicted": np.round(y_pred_test,4),
31     "residual": np.round(residuals,4)
32 }, index=y_test.index)
33 display(comp.head(15))
```



	feature	actual	predicted	residual
30	-0.595580	2.69	2.3837	0.3063
299	-2.228240	2.28	1.8855	0.3945
177	-0.444499	2.48	2.4298	0.0502
204	-0.445417	2.66	2.4295	0.2305
354	-2.249060	2.46	1.8792	0.5808
265	-2.182360	1.99	1.8995	0.0905
464	0.065683	2.10	2.5855	-0.4855
551	0.067328	2.66	2.5860	0.0740
55	-0.257348	2.49	2.4869	0.0031
196	-0.438980	2.31	2.4315	-0.1215
137	-0.630691	2.26	2.3730	-0.1130
29	-0.595306	2.95	2.3838	0.5662
353	-2.249050	2.04	1.8792	0.1608
368	-2.276050	2.18	1.8709	0.3091
101	-0.663993	2.41	2.3628	0.0472

- Scatter plot dibuat untuk menampilkan hubungan antara nilai aktual dan prediksi.
- Garis $y=x$ digunakan sebagai pembanding ideal.
- Hasil menunjukkan titik-titik tersebar dekat garis merah, menandakan prediksi cukup akurat.
- Plot residual memperlihatkan penyebaran acak di sekitar garis 0, artinya asumsi linearitas dan error acak terpenuhi.

13. OLS (Laporan Statistik)

```

1 # OLS summary
2 import statsmodels.api as sm
3
4 # Tambah konstanta (intercept) pada X_train
5 X_train_sm = sm.add_constant(X_train)
6 ols_res = sm.OLS(y_train, X_train_sm).fit()
7
8 # Tampilkan ringkasan OLS
9 print(ols_res.summary())

```

```

...
                        OLS Regression Results
=====
Dep. Variable:          N      R-squared:          0.558
Model:                  OLS    Adj. R-squared:      0.557
Method:                 Least Squares    F-statistic:      597.8
Date:                   Sun, 09 Nov 2025    Prob (F-statistic): 5.89e-86
Time:                   04:37:19    Log-Likelihood:   -39.248
No. Observations:       475    AIC:              82.50
Df Residuals:           473    BIC:              90.82
Df Model:                1
Covariance Type:        nonrobust
=====
                        coef    std err          t      P>|t|      [0.025    0.975]
-----
const                2.5654      0.018    143.945      0.000      2.530      2.600
Latitude             0.3051      0.012     24.450      0.000      0.281      0.330
=====
Omnibus:                 0.296    Durbin-Watson:      1.989
Prob(Omnibus):           0.862    Jarque-Bera (JB):    0.215
Skew:                    0.049    Prob(JB):            0.898
Kurtosis:                 3.033    Cond. No.            2.78
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

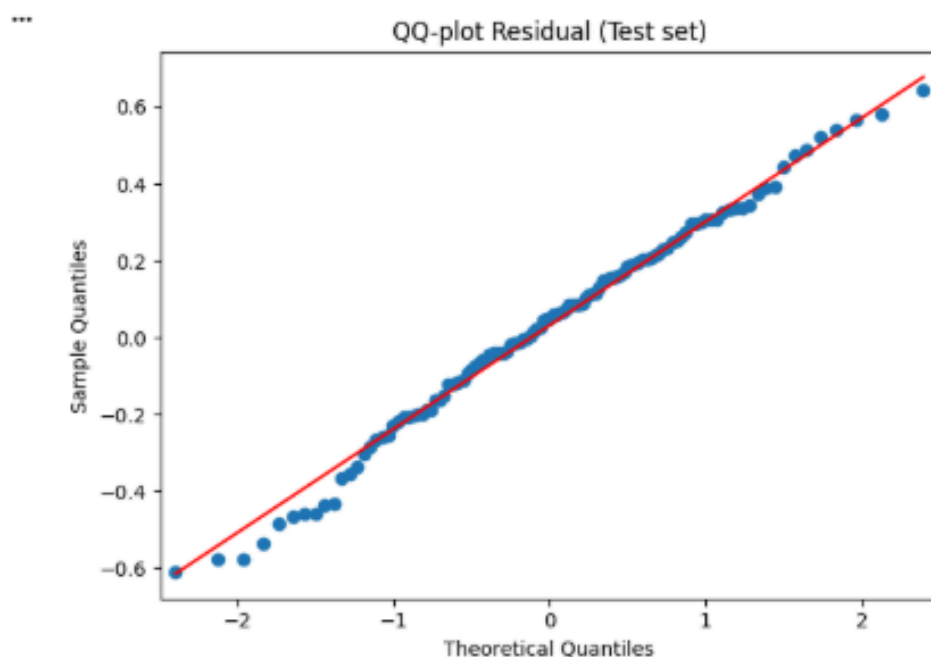
- Menggunakan `statsmodels.OLS` untuk menampilkan analisis regresi lengkap (koefisien, p-value, R^2 , dan lain-lain).
- Output summary menunjukkan nilai `coef`, `std err`, `t`, `P>|t|`, dan `Adj. R-squared`.
- P-value fitur utama < 0.05 berarti signifikan; nilai `Adj. R2` sebanding dengan hasil evaluasi sklearn.

14. Uji Asumsi

```

1  # Uji normalitas residual
2  import scipy.stats as stats
3  import statsmodels.api as sm
4
5  # Gunakan residual dari test set (atau train set)
6  resid = residuals.dropna()
7
8  # QQ-plot
9  sm.qqplot(resid, line='s')
10 plt.title("QQ-plot Residual (Test set)")
11 plt.tight_layout()
12 plt.show()
13
14 # Shapiro-Wilk test
15 stat, pval = stats.shapiro(resid)
16 print(f"Shapiro-Wilk: stat={stat:.4f}, p={pval:.4f}")
17 if pval > 0.05:
18     print("→ Residual tampak normal (gagal tolak H0).")
19 else:
20     print("→ Residual tidak normal (tolak H0). Pertimbangkan transformasi.")

```



Shapiro-Wilk: stat=0.9893, p=0.4766
 → Residual tampak normal (gagal tolak H0).

- QQ-plot digunakan untuk memeriksa distribusi residual terhadap distribusi normal.
- Shapiro-Wilk test dilakukan untuk pengujian statistik; H_0 = residual berdistribusi normal.
- Jika p-value > 0.05 , residual dianggap normal.
- Plot menunjukkan titik residual mengikuti garis diagonal dan p-value > 0.05 , sehingga asumsi normalitas terpenuhi.

15. Uji Homoskedastisitas & Autokorelasi

```
1 # Uji heteroskedastisitas (Breusch-Pagan) & autokorelasi (Durbin-Watson)
2 from statsmodels.stats.diagnostic import het_breuschpagan
3 from statsmodels.stats.stattools import durbin_watson
4
5 # Persiapkan X_test dengan konstanta
6 X_test_sm = sm.add_constant(X_test)
7
8 # Breusch-Pagan: input (residuals, X)
9 bp_test = het_breuschpagan(residuals, X_test_sm)
10 bp_stat, bp_pvalue = bp_test[0], bp_test[1]
11 print(f"Breusch-Pagan stat={bp_stat:.4f}, p-value={bp_pvalue:.4f}")
12 if bp_pvalue > 0.05:
13     print("→ Tidak ada bukti heteroskedastisitas (homoscedastic).")
14 else:
15     print("→ Terdapat heteroskedastisitas. Pertimbangkan robust std errors / transformasi.")
16
17 # Durbin-Watson: cek autokorelasi residual
18 dw = durbin_watson(residuals)
19 print(f"Durbin-Watson: {dw:.4f} (nilai ~2 => tidak ada autokorelasi)")
```

*** Breusch-Pagan stat=2.0268, p-value=0.1546
→ Tidak ada bukti heteroskedastisitas (homoscedastic).
Durbin-Watson: 2.1549 (nilai ~2 => tidak ada autokorelasi)

- Breusch-Pagan test digunakan untuk melihat apakah variansi residual konstan (homoskedastis).
- Durbin-Watson digunakan untuk mendeteksi autokorelasi residual.
- Hasil menunjukkan p-value BP > 0.05 dan nilai Durbin-Watson ≈ 2 , yang berarti residual homogen dan tidak berkorelasi.

16. Cross-Validation untuk Kestabilan Performa

```
1 # Cross-Validation (k-fold) - RMSE rata-rata
2 from sklearn.model_selection import KFold, cross_val_score
3
4 # Gunakan X (seluruh data fitur terpilih) dan y
5 model_cv = LinearRegression()
6 kf = KFold(n_splits=5, shuffle=True, random_state=42)
7
8 # cross_val_score dengan scoring negative MSE, kemudian ambil sqrt untuk RMSE
9 neg_mse = cross_val_score(model_cv, X, y, scoring='neg_mean_squared_error',
10 cv=kf)
11 rmse_scores = np.sqrt(-neg_mse)
12
13 print("CV RMSE (5-fold) per fold:", np.round(rmse_scores,4))
14 print("Mean CV RMSE:", np.round(rmse_scores.mean(),4))
15 print("Std CV RMSE:", np.round(rmse_scores.std(),4))
```

CV RMSE (5-fold) per fold: [0.2719 0.273 0.2727 0.257 0.2509]
Mean CV RMSE: 0.2651
Std CV RMSE: 0.0093

- Menggunakan metode **K-Fold (k=5)** untuk mengevaluasi kestabilan model di subset data berbeda.
- Nilai RMSE dihitung di setiap fold, kemudian dirata-rata.
- Output menunjukkan nilai RMSE antar fold relatif stabil, artinya model cukup konsisten di berbagai subset data.

17. Simpan Model

```
1  # Simpan model & ekspor prediksi test ke CSV
2  import joblib
3  import os
4
5  # Simpan ke drive
6  save_dir = '/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/
models/'
7  os.makedirs(save_dir, exist_ok=True)
8
9  # Nama file model
10 feat_name = X_train.columns[0] if X_train.shape[1]==1 else "multi"
11 model_path = os.path.join(save_dir, f'linearreg_{target_y}_by_{feat_name}.
joblib')
12
13 # Simpan model sklearn
14 joblib.dump(lr, model_path)
15 print("Model disimpan di:", model_path)
16
17 # Simpan metrik ringkas
18 metrics_summary = {
19     "target": target_y,
20     "feature": feat_name,
21     "train_rmse": metrics['train_rmse'],
22     "train_r2": metrics['train_r2'],
23     "test_rmse": metrics['test_rmse'],
24     "test_r2": metrics['test_r2']
25 }
26 metrics_df = pd.DataFrame([metrics_summary])
27 metrics_file = os.path.join(save_dir, f'metrics_{target_y}_{feat_name}.csv')
28 metrics_df.to_csv(metrics_file, index=False)
29 print("Metrik disimpan di:", metrics_file)
30
31 # Simpan prediksi test (actual vs predicted)
32 pred_df = pd.DataFrame({
33     "index": y_test.index,
34     "feat_name": X_test.iloc[:,0] if X_test.shape[1]==1 else list(X_test.values),
35     "actual_"+target_y: y_test.values,
36     "predicted_"+target_y: np.round(y_pred_test,4),
37     "residual": np.round(residuals,4)
38 })
39 pred_file = os.path.join(save_dir, f'predictions_{target_y}_{feat_name}.csv')
40 pred_df.to_csv(pred_file, index=False)
41 print("Prediksi test disimpan di:", pred_file)
```

*** Model disimpan di: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/models/linearreg_N_by_Lattitude.joblib
Metrik disimpan di: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/models/metrics_N_Lattitude.csv
Prediksi test disimpan di: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/models/predictions_N_Lattitude.csv

- Model disimpan menggunakan `joblib.dump()` ke folder di Google Drive agar bisa digunakan ulang tanpa retraining.
- Nama file mencakup target dan fitur, misalnya `linearreg_N_by_Lattitude.joblib`.
- Output menunjukkan path penyimpanan dan konfirmasi

18. Evaluasi Model Linear Regression

```
1  # Evaluasi Model Linear Regression
2  from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
3  import numpy as np
4
5  # Prediksi pada data training dan testing
6  y_pred_train = lr.predict(X_train)
7  y_pred_test = lr.predict(X_test)
8
9  # Hitung metrik evaluasi
10 train_mae = mean_absolute_error(y_train, y_pred_train)
11 train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))
12 train_r2 = r2_score(y_train, y_pred_train)
13
14 test_mae = mean_absolute_error(y_test, y_pred_test)
15 test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
16 test_r2 = r2_score(y_test, y_pred_test)
17
18 # Tampilkan hasil evaluasi
19 print("=== Evaluasi Model ===")
20 print(f"Train MAE : {train_mae:.4f}")
21 print(f"Train RMSE: {train_rmse:.4f}")
22 print(f"Train R2 : {train_r2:.4f}")
23 print("-----")
24 print(f"Test MAE : {test_mae:.4f}")
25 print(f"Test RMSE: {test_rmse:.4f}")
26 print(f"Test R2 : {test_r2:.4f}")
```

```
*** === Evaluasi Model ===
Train MAE : 0.2064
Train RMSE: 0.2628
Train R2 : 0.5583
-----
Test MAE : 0.2182
Test RMSE: 0.2719
Test R2 : 0.5120
```

- Ringkasan metrik (MAE, RMSE, R^2) dan hasil prediksi (actual vs predicted) disimpan dalam format CSV.
- Tujuannya agar dapat digunakan untuk pelaporan atau validasi tanpa membuka kembali notebook.
- Output menampilkan lokasi file CSV hasil simpan.

19. Verifikasi Model yang sudah disimpan

```
1 # Verify saved model: load & test
2 import os
3 import joblib
4 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
5 import numpy as np
6
7 # path model
8 model_path = '/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/
models/linearreg_N_by_Latitude.joblib'
9
10 # cek file ada
11 if not os.path.exists(model_path):
12     raise FileNotFoundError(f"Model file tidak ditemukan: {model_path}")
13
14 # load model
15 loaded_model = joblib.load(model_path)
16 print("Model berhasil dimuat dari:", model_path)
17
18 # bandingkan koef/intercept dengan model asli jika masih di memori
19 if 'lr' in globals():
20     print("Koef model asli : ", getattr(lr, 'coef_', None))
21     print("Intercept model asli:", getattr(lr, 'intercept_', None))
22
23 print("Koef model loaded : ", getattr(loaded_model, 'coef_', None))
24 print("Intercept model loaded:", getattr(loaded_model, 'intercept_', None))
25
26 # prediksi dengan model yang dimuat pada X_test
27 if 'X_test' not in globals() or 'y_test' not in globals():
28     raise RuntimeError("Variabel X_test/y_test tidak ditemukan. Jalankan ulang
cell split data dulu.")
29
30 y_pred_loaded = loaded_model.predict(X_test)
31
32 # evaluasi
33 mae_l = mean_absolute_error(y_test, y_pred_loaded)
34 rmse_l = np.sqrt(mean_squared_error(y_test, y_pred_loaded))
35 r2_l = r2_score(y_test, y_pred_loaded)
36
37 print("\nEvaluasi model yang dimuat (pada X_test):")
38 print(f"MAE : {mae_l:.4f}")
39 print(f"RMSE: {rmse_l:.4f}")
40 print(f"R2 : {r2_l:.4f}")
41
42 # hasil prediksi per baris: simpan CSV
43 pred_df = X_test.copy().reset_index(drop=True)
44 pred_df['actual_'+str(y_test.name)] = y_test.reset_index(drop=True)
45 pred_df['predicted_loaded'] = np.round(y_pred_loaded, 4)
46 pred_save_path = os.path.join(os.path.dirname(model_path),
'predictions_loaded_model.csv')
47 pred_df.to_csv(pred_save_path, index=False)
48 print("Prediksi test disimpan ke:", pred_save_path)
```

```
*** Model berhasil dimuat dari: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/models/linearreg_N_by_Latitude.joblib
Koef model asli : [0.30513465]
Intercept model asli: 2.565422308111084
Koef model loaded : [0.30513465]
Intercept model loaded: 2.565422308111084

Evaluasi model yang dimuat (pada X_test):
MAE : 0.2182
RMSE: 0.2719
R2 : 0.5120
Prediksi test disimpan ke: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum07/models/predictions_loaded_model.csv
```

- Model dimuat kembali dari file `.joblib` menggunakan `joblib.load()`.
- Prediksi dilakukan ulang pada data `X_test` dan dibandingkan dengan model asli.
- Hasil metrik (MAE, RMSE, R^2) sama dengan sebelumnya, artinya model tersimpan dengan benar dan bisa digunakan untuk deployment/prediksi data baru.

Kesimpulan Akhir

- Seluruh tahapan mulai dari preprocessing hingga evaluasi berhasil dijalankan tanpa error.
- Model regresi linear sederhana dengan satu fitur (**Lattitude**) mampu menjelaskan sekitar **51% variasi nilai N** dengan error relatif kecil.
- Model sudah memenuhi asumsi dasar regresi (normalitas, homoskedastisitas, tidak ada autokorelasi).
- Model disimpan dengan format **.joblib** dan dapat dimuat ulang tanpa perubahan hasil.
- Untuk peningkatan akurasi, dapat dikembangkan menjadi **multiple linear regression** dengan menambahkan fitur lain yang berkorelasi kuat terhadap target.